

Infrastructure as Code (IaC) with Terraform for AWS Environments

Overview of Project

Scenario:

A growing technology company manages its AWS infrastructure **manually through the AWS Console**. As the environment grows, this approach starts causing serious problems:

- Infrastructure changes are undocumented
- Environments drift over time
- Reproducing setups across accounts is difficult
- Rollbacks are risky and error-prone
- Collaboration between engineers is inconsistent

The engineering team wants a **reliable, repeatable, and version-controlled way** to manage AWS infrastructure.

My Role as the DevOps Engineer:

My Role was to design and build AWS infrastructure **entirely using Terraform**, following real-world DevOps best practices.

I was responsible for:

- Writing Terraform code to provision AWS resources
- Managing infrastructure state safely using a remote backend
- Building infrastructure incrementally instead of all at once
- Refactoring Terraform code into reusable modules
- Understanding how Terraform tracks, plans, and applies changes

Solution:

I used **Terraform** to provision and manage AWS infrastructure from scratch.

The solution includes:

- A Terraform project structured for real environments
- A **remote backend** using Amazon S3 and DynamoDB for state and locking
- Core AWS infrastructure built step by step
- Refactored Terraform code using **modules** for reusability
- Safe change management using terraform plan and terraform apply

About the Project:

In this hands-on project, I learnt how to manage AWS infrastructure **using code**, not the console.

I understood:

- How Terraform interacts with AWS APIs
- How Terraform state works and why remote state is critical
- How dependencies are resolved automatically
- How to safely modify infrastructure without accidental deletion

- How modular Terraform code is structured in real teams

By the end of this project, I was able to **read, write, and reason about Terraform code with confidence.**

Steps To Be Performed:

I completed the project in the following stages:

1. Understand and configure Terraform locally
2. Configured a **remote backend** using S3 and DynamoDB
3. Build AWS infrastructure incrementally using Terraform
4. Review and apply changes safely using terraform plan
5. Refactored infrastructure into **Terraform modules**

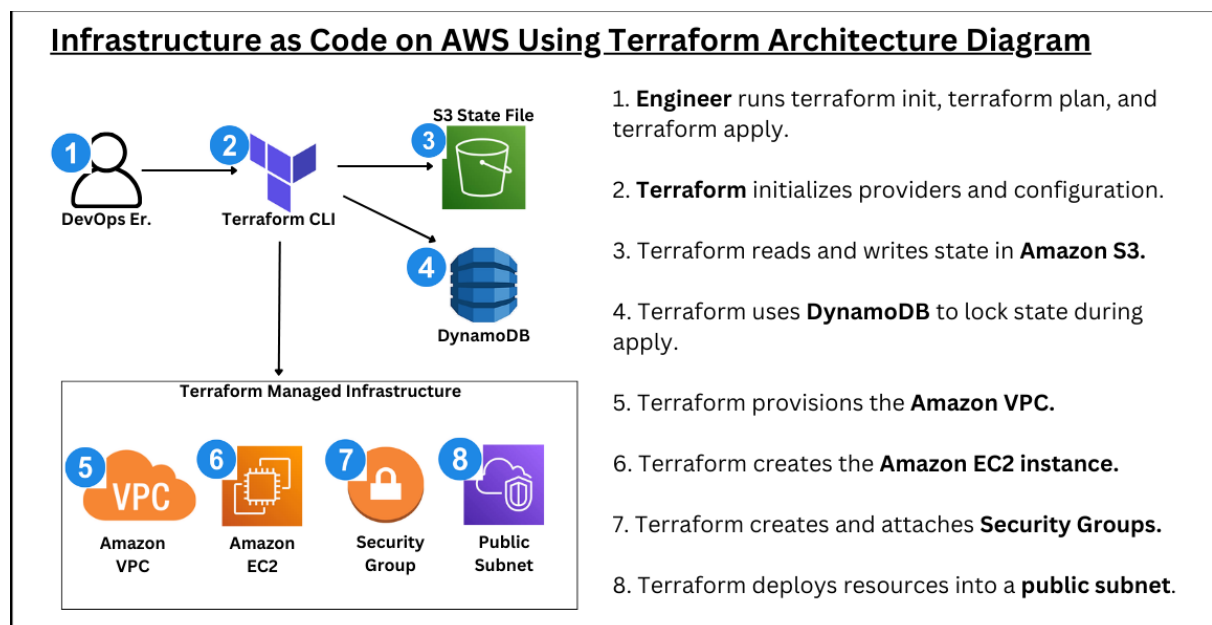
Each step focuses on **why Terraform behaves the way it does**, not just running commands.

Services Used:

- **Terraform** - Infrastructure as Code tool
- **Amazon S3** - Store Terraform remote state
- **Amazon DynamoDB** - State locking and consistency

- **Amazon VPC** - Networking foundation
- **Amazon EC2** - Compute resources
- **Security Groups** - Network access control
- **AWS IAM** - Permissions for Terraform operations

Architectural Diagram:



Final Result:

At the end of this project:

- AWS infrastructure created entirely using Terraform.
- Remote state management with locking.
- Modular, reusable Terraform code with a strong foundation for managing real-world cloud infrastructure.



Deployed with Terraform

Stage 1 — Local Environment Setup

Step 1.1 Installed AWS CLI:

Ran the appropriate installer for the OS and confirmed the installation:

```
PS C:\Users\rohan> aws --version
aws-cli/2.34.54 Python/3.14.5 Windows/11 exe/AMD64
PS C:\Users\rohan>
```

Step 1.2 Created a Dedicated IAM User for Terraform:

A dedicated IAM user with programmatic access gives Terraform only the permissions it needs — this is a security best practice in all production teams.

Navigated to IAM → Users → Create User and configured the following:

- **Username:** terraform-user
- **Access type:** Programmatic access only (CLI use)
- **Policy attached:** AdministratorAccess (scoped to least-privilege in production)
- **Access key:** Generated and downloaded the .csv file securely

Specify user details

User details

User name

terraform-user

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . @ _ - (hyphen)

☐ Provide user access to the AWS Management Console - *optional*

In addition to console access, users with `SignInLocalDevelopmentAccess` permissions can use the same console credentials for programmatic access without the need for access keys.

Permissions options

☐ **Add user to group**
Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐ **Copy permissions**
Copy all group memberships, attached managed policies, and inline policies from an existing user.

☒ **Attach policies directly**
Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

Permissions policies (1/1503)

Create policy

Choose one or more policies to attach to your new user.

AdministratorAccess

X

Filter by Type

All types

5 matches

< 1 >

⚙

Policy name	Type	Attached entities
AdministratorAccess	AWS managed - job function	0

[View user](#)

IAM users (1)

Info

Delete

Create user

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

< 1 >

⚙

☐	User name	▲	Path	▼	Group: ▼	Last activity	▼	MFA	▼	Password age	▼	Console last sign-in	▼	Access keys
☐	terraform-user		/		0	-		-		-		-		-

Step 1.3 Configured AWS CLI with IAM Credentials:

Running 'aws configure' stores credentials locally in ~/.aws/credentials. Terraform picks these up automatically.

```
PS C:\Users\rohan> aws configure
AWS Access Key ID [*****FWHY]: AKIARQHAPIGQVCWAGRWH
AWS Secret Access Key [*****CzSE]: IFT65ZbbTE6fHWDMMnpjLpL/+fobvHZdJVU/P6uc1
Default region name [ap-south-2]: ap-south-2
Default output format [json]: json
PS C:\Users\rohan>
```

The **aws sts get-caller-identity** command confirmed the credentials were valid — it returned the Account ID and the ARN of the **terraform-user** IAM user.

```
PS C:\Users\rohan> aws sts get-caller-identity
{
  "UserId": "AIDARQHAPIGQZHESXOABQ",
  "Account": "103549976993",
  "Arn": "arn:aws:iam::103549976993:user/terraform-user"
}
PS C:\Users\rohan>
```

Step 1.4 Installed Terraform:

Terraform is the core tool that reads .tf configuration files and translates them into AWS API calls. It needs to be installed locally before any infrastructure work can begin.

```
PS C:\Users\rohan> terraform version
Terraform v1.15.5
on windows_amd64
PS C:\Users\rohan>
```

Step 1.5 Created the Project Folder Structure:

A well-organized folder structure is how real DevOps teams maintain Terraform projects. Separating root config from reusable modules makes the code readable, testable, and reusable across environments.

```
mkdir -p modules/vpc
mkdir -p modules/ec2
mkdir -p modules/security-group
```

```
touch main.tf variables.tf outputs.tf provider.tf backend.tf

touch modules/vpc/main.tf
touch modules/vpc/variables.tf
touch modules/vpc/outputs.tf

touch modules/ec2/main.tf
touch modules/ec2/variables.tf
touch modules/ec2/outputs.tf

touch modules/security-group/main.tf
touch modules/security-group/variables.tf
touch modules/security-group/outputs.tf
```


The final project layout:

```
PS C:\Users\rohan\terraform-aws-project> tree /f
Folder PATH listing for volume Windows-SSD
Volume serial number is BE93-E1E9
C:.
├── backend.tf
├── main.tf
├── outputs.tf
├── provider.tf
├── variables.tf
└── modules
    ├── ec2
    │   ├── main.tf
    │   ├── outputs.tf
    │   └── variables.tf
    ├── security-group
    │   ├── main.tf
    │   ├── outputs.tf
    │   └── variables.tf
    └── vpc
        ├── main.tf
        ├── outputs.tf
        └── variables.tf
```

Stage 2 — Remote Backend — S3 & DynamoDB

By default, Terraform saves state to a local **terraform.tfstate** file. This works for solo projects but breaks in teams — two engineers applying simultaneously can corrupt the state file. The solution is a remote backend: **S3 stores the state** and **DynamoDB locks it** during apply.

Step 2.1 Created the S3 Bucket for State Storage:

The S3 bucket is the central store for the Terraform state file. Versioning is enabled so previous state versions can be recovered if something goes wrong. The bucket is created manually — Terraform cannot create its own backend.

Navigated to S3 → Create Bucket and configured:

- **Bucket name:** terraform-state-<my-account-id> (globally unique)
- **Region:** ap-south-2
- **Block all public access:** Enabled (security requirement)
- **Versioning:** Enabled (allows recovery of old state files)
- **Encryption:** SSE-S3 (server-side encryption, default)

General configuration

AWS Region
Asia Pacific (Hyderabad) ap-south-2

Bucket namespace

Choose the namespace where you want to create your bucket. [Learn more](#)

- ☒ **Global namespace**
By default, S3 creates general purpose buckets in the global namespace.
- ☐ **Account Regional namespace (recommended)**
General purpose buckets created in your account Regional namespace are unique to your account. These buckets can never be created by another AWS account.

Bucket name [Info](#)

terraform-state-103549976993

Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn more](#)

Copy settings from existing bucket - *optional*

Only the bucket settings in the following configuration are copied.

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of without public access. If you require some level of public access to this bucket or objects within, you can customize the individual settings below to suit you

- ☒ **Block *all* public access**
- Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, can easily recover from both unintended user actions and application failures. [Learn more](#)

☐ Disable

☒ Enable

Server-side encryption is automatically applied to new objects stored in this bucket.

Secure your objects with two separate layers of encryption. For details on pricing, see [DSSE-KMS pricing](#) on the **Storage** tab of the [Amazon S3 pricing page](#)

- ☒ Server-side encryption with Amazon S3 managed keys (SSE-S3)
- ☐ Server-side encryption with AWS Key Management Service keys (SSE-KMS)
- ☐ Dual-layer server-side encryption with AWS Key Management Service keys (DSSE-KMS)

✔ Successfully created bucket "terraform-state-103549976993"

Step 2.2 Created the DynamoDB Table for State Locking:

DynamoDB acts as a distributed lock. When 'terraform apply' starts, it writes a lock entry to the table. Any other apply attempt sees the lock and waits. This prevents two engineers from making conflicting infrastructure changes at the same time.

Navigated to DynamoDB → Create Table with these settings:

- **Table name:** terraform-state-lock
- **Partition key:** LockID (type: String) — exact name required by Terraform
- **Settings:** Default (free tier eligible — 25 RCU/WCU)

Create table

Table details [Info](#)
DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name
This will be used to identify your table.

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key
The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

1 to 255 characters and case sensitive.

[Alt+S] Ask Amazon Q

Asia Pacific (Hyderabad) Rohan Irkullawar (103549976993) Rohan Irkullawar

The terraform-state-lock table was created successfully.

Notifications 0 0 1 1 0

Tables (1/1) [Info](#) Last updated June 10, 2026, 11:41 (UTC+5:30) [Actions](#) [Delete](#) [Create table](#)

Find tables Filter by tag Any tag key Filter by tag value Any tag value

☒	Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity mode
☒	terraform-state-lock	Active	LockId (S)	-	0	0	Off	☆	On-demand

Step 2.3 Wrote the Backend Configuration:

The backend.tf file tells Terraform where to store and retrieve state. Every team member pointing to the same S3 key means everyone works from the same infrastructure state.

backend.tf:

```
Windows PowerShell
terraform {
  backend "s3" {
    bucket     = "terraform-state-103549976993"
    key        = "project/terraform.tfstate"
    region     = "ap-south-2"
    dynamodb_table = "terraform-state-lock"
    encrypt    = true
  }
}
```

provider.tf:

```
Windows PowerShell
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
  required_version = ">= 1.3.0"
}

provider "aws" {
  region = "ap-south-2"
}
```

Step 2.4 Initialized Terraform:

'terraform init' downloads the AWS provider plugin, connects to the S3 backend, and sets up the working directory. This must be run before any other Terraform command and whenever the backend or provider config changes.

Terraform confirmed the backend was configured successfully and downloaded the AWS provider plugin. The **.terraform/** directory was created with the provider binaries.

```
The parameter 'dynamodb_table' is deprecated. Use parameter 'use_lockfile' instead.
Terraform has been successfully initialized!
```

Stage 3 — Building AWS Infrastructure Incrementally:

Infrastructure was built in layers — VPC first (network foundation), then Security Group (access control), then EC2 (compute). Terraform's dependency graph automatically resolved the correct creation order.

Step 3.1 Wrote the VPC Module:

The VPC is the network foundation. Every AWS resource (EC2, RDS, Lambda) must live inside a VPC. The public subnet, internet gateway, and route table are required for the EC2 instance to be reachable from the internet.

modules/vpc/main.tf — resources created:

- **aws_vpc** — the isolated virtual network (CIDR: 10.0.0.0/16)
- **aws_internet_gateway** — gateway for internet access
- **aws_subnet** — public subnet in ap-south-2 (CIDR: 10.0.1.0/24)
- **aws_route_table** — routes 0.0.0.0/0 traffic to the internet gateway
- **aws_route_table_association** — links the subnet to the route table

```
resource "aws_vpc" "main" {
  cidr_block      = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support = true

  tags = {
    Name        = "${var.project_name}-vpc"
    Environment = var.environment
  }
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "${var.project_name}-igw"
  }
}

resource "aws_subnet" "public" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = var.public_subnet_cidr
  availability_zone  = var.availability_zone
  map_public_ip_on_launch = true
}
```

```

tags = {
  Name = "${var.project_name}-igw"
}

resource "aws_subnet" "public" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = var.public_subnet_cidr
  availability_zone = var.availability_zone
  map_public_ip_on_launch = true

  tags = {
    Name = "${var.project_name}-public-subnet"
  }
}

```

```

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }

  tags = {
    Name = "${var.project_name}-public-rt"
  }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}

```

Step 3.2 Wrote the Security Group Module:

Security Groups are AWS's virtual firewall. Without explicit inbound rules, no traffic reaches the EC2 instance. The rules below allow SSH (port 22) for management and HTTP (port 80) for the web server — both required for this project.

modules/security-group/main.tf — rules configured:

- **Inbound port 22 (SSH):** For connecting to the EC2 instance via terminal
- **Inbound port 80 (HTTP):** For accessing the Apache web server
- **Outbound all:** Allow EC2 to reach the internet (updates, installs, etc.)

```

resource "aws_security_group" "web" {
  name     = "${var.project_name}-sg"
  vpc_id   = var.vpc_id

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

```

Step 3.3 Wrote the EC2 Module:

EC2 is the virtual machine running inside the VPC. The 't2.micro' instance type is used to stay within Free Tier limits. The AMI is fetched dynamically so the code always uses the latest Amazon Linux 2 — no hardcoded AMI IDs. A user_data script bootstraps Apache on first boot.

```

data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name     = "name"
    values   = ["amzn2-ami-hvm-*x86_64-gp2"]
  }
}

resource "aws_instance" "web" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t3.micro"
  subnet_id     = var.subnet_id
  vpc_security_group_ids = [var.security_group_id]
  key_name      = var.key_name

  user_data = <<-EOF
  #!/bin/bash
  yum update -y
  yum install -y httpd
  systemctl start httpd
  echo "<h1>Deployed with Terraform</h1>" > /var/www/html/index.html
  EOF
}

```

Step 3.4 Generated an EC2 Key Pair:

An SSH key pair is required to connect to the EC2 instance. The private key stays on the local machine; the public key is uploaded to AWS. Terraform reads the public key file and creates the key pair resource automatically.

```
PS C:\Users\rohan\terraform-aws-project> ssh-keygen -t rsa -b 2048 -f $HOME\.ssh\terraform-key
Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\rohan\.ssh\terraform-key
Your public key has been saved in C:\Users\rohan\.ssh\terraform-key.pub
The key fingerprint is:
SHA256:0ItSkMci+OHraeP5mLKQZrD0c9e8skq7j4wzk14SwSg rohan@Rohanxrico
The key's randomart image is:
+---[RSA 2048]-----+
|  .  .o               |
| o.= o.o.             |
| E*.= oo +            |
| =ooo.o + +           |
| o. o+ o S .          |
|  o .. .              |
| o o.o o              |
| . =B* o.             |
| +**=**+             |
+---[SHA256]-----+
```

```
Windows PowerShell
resource "aws_key_pair" "deployer" {
  key_name      = "terraform-key"
  public_key    = file("C:/Users/rohan/.ssh/terraform-key.pub")
}
```

Step 3.5 Wrote the Root main.tf:

The root main.tf is the entry point that wires all modules together. Terraform automatically builds a dependency graph and creates resources in the correct order.

```
Windows PowerShell
resource "aws_key_pair" "deployer" {
  key_name      = "terraform-key"
  public_key    = file("C:/Users/rohan/.ssh/terraform-key.pub")
}

module "vpc" {
  source          = "../modules/vpc"
  project_name    = var.project_name
  environment     = var.environment
  vpc_cidr        = "10.0.0.0/16"
  public_subnet_cidr = "10.0.1.0/24"
  availability_zone = "ap-south-2a"
}

module "security_group" {
  source          = "../modules/security-group"
  project_name    = var.project_name
  vpc_id          = module.vpc.vpc_id # Dependency: VPC must exist first
}

module "ec2" {
  source          = "../modules/ec2"
  project_name    = var.project_name
  environment     = var.environment
  subnet_id       = module.vpc.public_subnet_id # From VPC module output
  security_group_id = module.security_group.security_group_id
  key_name        = aws_key_pair.deployer.key_name
  instance_type   = "t2.micro"
}
```


Stage 4 — Plan, Apply & Verify

Step 4.1 Formatted and Validated the Code:

'terraform fmt' enforces consistent formatting across the team — no style debates.

'terraform validate' checks for syntax errors and config issues without touching AWS.

Running both before plan catches problems early.

```
Windows PowerShell
PS C:\Users\rohan\terraform-aws-project> terraform fmt -recursive
PS C:\Users\rohan\terraform-aws-project> terraform validate
Success! The configuration is valid.

PS C:\Users\rohan\terraform-aws-project> |
```

Step 4.2 Ran Terraform Plan:

'terraform plan' is the most important command in any Terraform workflow. It contacts AWS to compare the current state of real infrastructure against what the .tf files describe — then shows exactly what will be created, changed, or destroyed WITHOUT making any changes.

Reviewing the plan before applying is non-negotiable in production.

```
Windows PowerShell
PS C:\Users\rohan\terraform-aws-project> terraform plan -out=tfplan

Warning: Deprecated Parameter
The parameter "dynamodb_table" is deprecated. Use parameter "use_lockfile" instead.

module.ec2.data.aws_ami.amazon_linux: Reading...
module.ec2.data.aws_ami.amazon_linux: Read complete after 0s [id=ami-0eb54ba25637f46db]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# aws_key_pair.deployer will be created
+ resource "aws_key_pair" "deployer" {
+   arn           = (known after apply)
+   fingerprint  = (known after apply)
+   id           = (known after apply)
+   key_name      = "terraform-key"
+   key_name_prefix = (known after apply)
+   key_pair_id   = (known after apply)
+   key_type      = (known after apply)
+   public_key    = "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDdV6HhWMS9KY/gkSB8UWUuRGbQ3tVel+zs98zL51l47c-f6W2JAEHkxWNWkgQYt0AJYU1Q1LF1xW6LyfRyiI1sdBk81EP
01LAcbA28JYVFLYRpFzTthhNMueSVtZlpPha6MPaVM5hI33M+E4qLwunX2DuRTIDP:P5bqdf/SY1xqILy4wPA1UyVTL2y2uvcM/ZvVqJZ4ZJvYt+Mp5JpRCqWabA041MmoLGLYURPu27eeSDdId+5BnrFvX5
DGIcWxSQwSSEMyeokr63lb5gPWvFTBeyeloPJzntiu+B0c1Zo78UDkIZ9JL8T5WwadwZQ2F3LNSlIdN60W2G2F7LJzgb rohan@Rohanxrico"
+   tags_all      = (known after apply)
}
```

```
Windows PowerShell

# module.ec2.aws_instance.web will be created
+ resource "aws_instance" "web" {
+   ami                      = "ami-9eb54ba25637f46db"
+   arn                      = (known after apply)
+   associate_public_ip_address = (known after apply)
+   availability_zone        = (known after apply)
+   cpu_core_count           = (known after apply)
+   cpu_threads_per_core     = (known after apply)
+   disable_api_stop         = (known after apply)
+   disable_api_termination  = (known after apply)
+   ebs_optimized            = (known after apply)
+   enable_primary_ipv6      = (known after apply)
+   get_password_data        = false
+   host_id                  = (known after apply)
+   host_resource_group_arn   = (known after apply)
+   iam_instance_profile     = (known after apply)
+   id                       = (known after apply)
+   instance_initiated_shutdown_behavior = (known after apply)
+   instance_lifecycle       = (known after apply)
+   instance_state           = (known after apply)
+   instance_type            = "t2.micro"
+   ipv6_address_count       = (known after apply)
+   ipv6_addresses           = (known after apply)
+   key_name                 = "terraform-key"
+   monitoring               = (known after apply)
+   outpost_arn              = (known after apply)
+   password_data            = (known after apply)
+   placement_group          = (known after apply)
+   placement_partition_number = (known after apply)
+   primary_network_interface_id = (known after apply)
+   private_dns              = (known after apply)
+   private_ip               = (known after apply)
+   public_dns               = (known after apply)
+   public_ip                = (known after apply)
+   secondary_private_ips    = (known after apply)
+   security_groups          = (known after apply)
+   source_dest_check        = true
+   spot_instance_request_id = (known after apply)
+   subnet_id                = (known after apply)
+   tags_all                 = (known after apply)
}
```

```
Windows PowerShell

+ tags_all                 = (known after apply)
+ tenancy                  = (known after apply)
+ user_data                = "a117eeef5757bd07d233e17e3b0f0bb17c9393e2"
+ user_data_base64        = (known after apply)
+ user_data_replace_on_change = false
+ vpc_security_group_ids   = (known after apply)

+ capacity_reservation_specification (known after apply)

+ cpu_options (known after apply)

+ ebs_block_device (known after apply)

+ enclave_options (known after apply)

+ ephemeral_block_device (known after apply)

+ instance_market_options (known after apply)

+ maintenance_options (known after apply)

+ metadata_options (known after apply)

+ network_interface (known after apply)

+ private_dns_name_options (known after apply)

+ root_block_device (known after apply)
}
```

```
Windows PowerShell

# module.security_group.aws_security_group.web will be created
+ resource "aws_security_group" "web" {
+   arn                = (known after apply)
+   description        = "Managed by Terraform"
+   egress              = [
+     {
+       cidr_blocks      = [
+         "0.0.0.0/0",
+       ]
+       from_port         = 0
+       ipv6_cidr_blocks  = []
+       prefix_list_ids   = []
+       protocol          = "-1"
+       security_groups   = []
+       self              = false
+       to_port           = 0
+       # (1 unchanged attribute hidden)
+     },
+   ]
+   id                  = (known after apply)
+   ingress              = [
+     {
+       cidr_blocks      = [
+         "0.0.0.0/0",
+       ]
+       from_port         = 22
+       ipv6_cidr_blocks  = []
+       prefix_list_ids   = []
+       protocol          = "tcp"
+       security_groups   = []
+       self              = false
+       to_port           = 22
+       # (1 unchanged attribute hidden)
+     },
+     {
+       cidr_blocks      = [
+         "0.0.0.0/0",
+       ]
+       from_port         = 80
+       ipv6_cidr_blocks  = []

```

```
Windows PowerShell

+   prefix_list_ids    = []
+   protocol            = "tcp"
+   security_groups     = []
+   self               = false
+   to_port             = 80
+   # (1 unchanged attribute hidden)
+ },
+ ],
+ name                  = "terraform-aws-project-sg"
+ name_prefix           = (known after apply)
+ owner_id              = (known after apply)
+ revoke_rules_on_delete = false
+ tags_all              = (known after apply)
+ vpc_id                = (known after apply)
+ }

# module.vpc.aws_internet_gateway.main will be created
+ resource "aws_internet_gateway" "main" {
+   arn                = (known after apply)
+   id                 = (known after apply)
+   owner_id           = (known after apply)
+   tags               = {
+     "Name" = "terraform-aws-project-igw"
+   }
+   tags_all           = {
+     "Name" = "terraform-aws-project-igw"
+   }
+   vpc_id             = (known after apply)
+ }

# module.vpc.aws_route_table.public will be created
+ resource "aws_route_table" "public" {
+   arn                = (known after apply)
+   id                 = (known after apply)
+   owner_id           = (known after apply)
+   propagating_vgws   = (known after apply)
+   route              = [
+     {
+       cidr_block       = "0.0.0.0/0"
+       gateway_id        = (known after apply)
+       # (11 unchanged attributes hidden)

```

```
Windows PowerShell x + v
# (11 unchanged attributes hidden)
    },
  ],
+ tags              = {
+   + "Name" = "terraform-aws-project-public-rt"
+ }
+ tags_all          = {
+   + "Name" = "terraform-aws-project-public-rt"
+ }
+ vpc_id            = (known after apply)
}

# module.vpc.aws_route_table_association.public will be created
+ resource "aws_route_table_association" "public" {
+   id              = (known after apply)
+   route_table_id = (known after apply)
+   subnet_id       = (known after apply)
+ }

# module.vpc.aws_subnet.public will be created
+ resource "aws_subnet" "public" {
+   arn                                = (known after apply)
+   assign_ipv6_address_on_creation    = false
+   availability_zone                  = "us-east-1a"
+   availability_zone_id               = (known after apply)
+   cidr_block                         = "10.0.1.0/24"
+   enable_dns64                       = false
+   enable_resource_name_dns_a_record_on_launch = false
+   enable_resource_name_dns_aaaa_record_on_launch = false
+   id                                 = (known after apply)
+   ipv6_cidr_block_association_id     = (known after apply)
+   ipv6_native                        = false
+   map_public_ip_on_launch            = true
+   owner_id                           = (known after apply)
+   private_dns_hostname_type_on_launch = (known after apply)
+   tags                               = {
+     + "Name" = "terraform-aws-project-public-subnet"
+   }
+   tags_all                           = {
+     + "Name" = "terraform-aws-project-public-subnet"
+   }
+ }
```

```
Windows PowerShell x + v
# module.vpc.aws_vpc.main will be created
+ resource "aws_vpc" "main" {
+   arn                                = (known after apply)
+   cidr_block                         = "10.0.0.0/16"
+   default_network_acl_id             = (known after apply)
+   default_route_table_id             = (known after apply)
+   default_security_group_id          = (known after apply)
+   dhcp_options_id                   = (known after apply)
+   enable_dns_hostnames               = true
+   enable_dns_support                 = true
+   enable_network_address_usage_metrics = (known after apply)
+   id                                 = (known after apply)
+   instance_tenancy                   = "default"
+   ipv6_association_id                = (known after apply)
+   ipv6_cidr_block                    = (known after apply)
+   ipv6_cidr_block_network_border_group = (known after apply)
+   main_route_table_id                = (known after apply)
+   owner_id                           = (known after apply)
+   tags                               = {
+     + "Environment" = "dev"
+     + "Name"        = "terraform-aws-project-vpc"
+   }
+   tags_all                           = {
+     + "Environment" = "dev"
+     + "Name"        = "terraform-aws-project-vpc"
+   }
+ }

Plan: 8 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ ec2_public_ip = (known after apply)
+ vpc_id        = (known after apply)
+ web_url       = (known after apply)

Saved the plan to: tfplan

To perform exactly these actions, run the following command to apply:
  terraform apply "tfplan"
```

Step 4.3 Applied the Infrastructure:

'terraform apply' executes the plan and makes real API calls to AWS. Using the saved plan file ensures exactly what was reviewed gets deployed

```

Saved the plan to: tfplan

To perform exactly these actions, run the following command to apply:
  terraform apply "tfplan"
PS C:\Users\rohan\terraform-aws-project> terraform apply "tfplan"

Warning: Deprecated Parameter
The parameter "dynamodb_table" is deprecated. Use parameter "use_lockfile" instead.

module.ec2.aws_instance.web: Creating...
module.ec2.aws_instance.web: Still creating... [00m10s elapsed]
module.ec2.aws_instance.web: Creation complete after 12s [id=i-0db349601a480c627]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:
ec2_public_ip = "98.130.122.209"
vpc_id = "vpc-03746d4f9f52fc6dd"
web_url = "http://98.130.122.209"
PS C:\Users\rohan\terraform-aws-project> |

```

Step 4.4 Verified All Resources in the AWS Console:

After every apply, it's good practice to visually verify the resources in the Console — confirming that resource IDs match Terraform's output and the infrastructure is configured exactly as intended.

Each resource was verified individually:

- **VPC:** Confirmed VPC ID matched Terraform output, CIDR 10.0.0.0/16, DNS hostnames enabled
- **Subnet:** Public subnet in ap-south-2a with CIDR 10.0.1.0/24, auto-assign public IP enabled
- **EC2 Instance:** t3.micro instance in 'running' state, public IP assigned, attached to correct subnet and security group
- **Security Group:** Inbound rules for ports 22 and 80 confirmed, outbound all traffic allowed
- **S3 Bucket:** The project/terraform.tfstate file appeared inside the bucket, confirming remote state was saved
- **Web Server:** Opened browser to <http://<ec2-public-ip>> and confirmed the Apache page displayed 'Deployed with Terraform on AWS'

Search

[Alt+S]

Asia Pacific (Hyderabad)

Rohan Irkullawar (103549976993)

Rohan Irkullawar

VPC

Your VPCs

vpc-03746d4f9f52fc6dd

Actions

VPC dashboard

AWS Global View

Filter by VPC

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

DHCP option sets

Elastic IPs

Managed prefix lists

NAT gateways

Peering connections

Route servers

Security

Network ACLs

vpc-03746d4f9f52fc6dd / terraform-aws-project-vpc

Details

Info

VPC ID

vpc-03746d4f9f52fc6dd

DNS resolution

Enabled

Main network ACL

acl-0f9f4b36261af3dcc

IPv6 CIDR

-

Encryption control ID

-

State

Available

Tenancy

default

Default VPC

No

Network Address Usage metrics

Disabled

Encryption control mode

-

Block Public Access

Off

DHCP option set

dopt-0d9a8c1c2c72c50d3

IPv4 CIDR

10.0.0.0/16

Route 53 Resolver DNS Firewall rule groups

-

DNS hostnames

Enabled

Main route table

rtb-0c4707e58f7dd351e

IPv6 pool

-

Owner ID

103549976993

Resource map

CIDRs

Flow logs

Tags

Integrations

IPv4 CIDRs

Info

Address family

IPv4

CIDR

10.0.0.0/16

Status

Associated

Edit CIDRs

EC2

Instances

i-0db349601a480c627

Connect

Instance state

Actions

EC2

Dashboard

AWS Global View

Events

Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Instance summary for i-0db349601a480c627

Info

Updated less than a minute ago

Instance ID

i-0db349601a480c627

IPv6 address

-

Hostname type

IP name: ip-10-0-1-66.ap-south-2.compute.internal

Answer private resource DNS name

-

Auto-assigned IP address

98.130.122.209 [Public IP]

IAM role

-

Public IPv4 address

98.130.122.209 | open address

Instance state

Running

Private IP DNS name (IPv4 only)

ip-10-0-1-66.ap-south-2.compute.internal

Instance type

t3.micro

VPC ID

vpc-03746d4f9f52fc6dd (terraform-aws-project-vpc)

Subnet ID

subnet-0a12df789e44fd17b (terraform-aws-project-public-subnet)

Private IPv4 addresses

10.0.1.66

Public DNS

ec2-98-130-122-209.ap-south-2.compute.amazonaws.com | open address

Elastic IP addresses

-

AWS Compute Optimizer finding

Opt-in to AWS Compute Optimizer for recommendation s. | Learn more

Auto Scaling Group name

-

sg-0e28cb7bdd731ee65 - terraform-aws-project-sg

[Actions](#)

Details

Security group name
terraform-aws-project-sg

Owner
103549976993

Security group ID
sg-0e28cb7bdd731ee65

Inbound rules count
2 Permission entries

Description
Managed by Terraform

Outbound rules count
1 Permission entry

VPC ID
vpc-03746d4f9f52fc6dd

[Inbound rules](#) | [Outbound rules](#) | [Sharing](#) | [VPC associations](#) | [Related resources - new](#) | [Tags](#)

Inbound rules (2)

[Manage tags](#) | [Edit inbound rules](#)

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	
☐	-	sgr-0f8bf89f15a1e7422	IPv4	SSH	TCP	22	C
☐	-	sgr-0764519e5d835a53f	IPv4	HTTP	TCP	80	C

[Alt+S]

Asia Pacific (Hyderabad)

Rohan Irkullawar (103549976993)

Rohan Irkullawar

-0a12df789e4afd17b

subnet-0a12df789e4afd17b / terraform-aws-project-public-subnet

Actions

Details

Subnet ID

subnet-0a12df789e4afd17b

IPv4 CIDR

10.0.1.0/24

Availability Zone

aps2-az1 (ap-south-2a)

Default subnet

No

IPv6 CIDR reservations

-

Resource name DNS AAAA record

Disabled

Subnet ARN

arn:aws:ec2:ap-south-2:103549976993:subnet/subnet-0a12df789e4afd17b

Available IPv4 addresses

250

VPC

vpc-03746d4f9f52fc6dd | terraform-aws-project-vpc

Auto-assign public IPv4 address

Yes

IPv6-only

No

DNS64

State

Available

IPv6 CIDR

-

Route table

rtb-076c73a82ade09dd6 | terraform-aws-project-public-rt

Auto-assign IPv6 address

No

Hostname type

IP name

Owner

103549976993

Block Public Access

Off

IPv6 CIDR association ID

-

Network ACL

acl-0f9f4b36261af3dcc

IPv4 CIDR reservations

-

Resource name DNS A record

Disabled

aws

Search

[Alt+S]

Asia Pacific (Hyderabad)

Rohan Irkullawar (103549976993)

Rohan Irkullawar

Amazon S3 > Buckets > terraform-state-103549976993 > project/

Amazon S3

Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Files

File systems

Access management and security

Access Points

Access Points for FSx

project/

Copy S3 URI

Objects

Properties

Objects (1)

Copy S3 URI

Copy URL

Download

Open L

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

Show versions

1

Name

Type

Last modified

Size

Storage class

terraform.tfstate

tfstate

June 10, 2026, 13:01:25 (UTC+05:30)

19.0 KB

Standard

←

→

↺

Not secure

98.130.122.209

☆

All Bookmarks

Deployed with Terraform

Stage 5 — Terraform State & Safe Change Management

Step 5.1 Inspected Terraform State:

Terraform state is its memory — the mapping between .tf code and real AWS resource IDs. Understanding state is critical: deleting the state file means Terraform loses track of existing resources and may try to recreate them (causing duplicates or errors).

```
PS C:\Users\rohan\terraform-aws-project> terraform state list
aws_key_pair.deployer
module.ec2.data.aws_ami.amazon_linux
module.ec2.aws_instance.web
module.security_group.aws_security_group.web
module.vpc.aws_internet_gateway.main
module.vpc.aws_route_table.public
module.vpc.aws_route_table_association.public
module.vpc.aws_subnet.public
module.vpc.aws_vpc.main
PS C:\Users\rohan\terraform-aws-project> |
```

Step 5.2 Made a Safe In-Place Change:

This step demonstrates the core Terraform change workflow — plan first, then apply. Adding a tag to the EC2 instance is an in-place update (no downtime, no resource replacement). The plan clearly shows '~ update in-place' so there are no surprises.

Added a **ManagedBy = Terraform** tag to the EC2 instance in **modules/ec2/main.tf**, then planned and applied:

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name   = "name"
    values = ["amzn2-ami-hvm-*x86_64-gp2"]
  }
}

resource "aws_instance" "web" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t3.micro"
  subnet_id     = var.subnet_id
  vpc_security_group_ids = [var.security_group_id]
  key_name      = var.key_name

  tags = {
    Name        = "${var.project_name}-web-server"
    Environment = var.environment
    ManagedBy   = "Terraform"
  }

  user_data = <<-EOF
  #!/bin/bash
  yum update -y
  yum install -y httpd
  systemctl start httpd
  echo "<h1>Deployed with Terraform</h1>" > /var/www/html/index.html
  EOF
}
```



```

PS C:\Users\rohan\terraform-aws-project> terraform apply "tfplan"

Warning: Deprecated Parameter
  The parameter "dynamodb_table" is deprecated. Use parameter "use_lockfile" instead.
module.ec2.aws_instance.web: Modifying... [id=i-0db349601a480c627]
module.ec2.aws_instance.web: Modifications complete after 1s [id=i-0db349601a480c627]

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

Outputs:
ec2_public_ip = "98.130.122.209"
vpc_id = "vpc-03746d4f9f52fc6dd"
web_url = "http://98.130.122.209"

```

Stage 6 — Cleanup & Resource Destruction

Step 6.1 Destroyed All Infrastructure

'terraform destroy' tears down every resource Terraform created, in reverse dependency order (EC2 before subnet, subnet before VPC). This is the clean, safe way to remove infrastructure — it prevents orphaned resources and avoids unexpected AWS charges on the Free Tier.

```

Enter a value: yes
module.vpc.aws_route_table_association.public: Destroying... [id=rtbassoc-01cea1631877c92a0]
module.ec2.aws_instance.web: Destroying... [id=i-0db349601a480c627]
module.vpc.aws_route_table_association.public: Destruction complete after 0s
module.vpc.aws_route_table.public: Destroying... [id=rtb-076c73a82ade09dd6]
module.vpc.aws_route_table.public: Destruction complete after 1s
module.vpc.aws_internet_gateway.main: Destroying... [id=igw-03fc555de42943798]
module.ec2.aws_instance.web: Still destroying... [id=i-0db349601a480c627, 00m10s elapsed]
module.vpc.aws_internet_gateway.main: Still destroying... [id=igw-03fc555de42943798, 00m10s elapsed]
module.vpc.aws_internet_gateway.main: Destruction complete after 17s
module.ec2.aws_instance.web: Still destroying... [id=i-0db349601a480c627, 00m20s elapsed]
module.ec2.aws_instance.web: Still destroying... [id=i-0db349601a480c627, 00m30s elapsed]
module.ec2.aws_instance.web: Destruction complete after 30s
aws_key_pair.deployer: Destroying... [id=terraform-key]
module.vpc.aws_subnet.public: Destroying... [id=subnet-0a12df789e4afd17b]
module.security_group.aws_security_group.web: Destroying... [id=sg-0e28cb7bdd731ee65]
aws_key_pair.deployer: Destruction complete after 0s
module.vpc.aws_subnet.public: Destruction complete after 0s
module.security_group.aws_security_group.web: Destruction complete after 1s
module.vpc.aws_vpc.main: Destroying... [id=vpc-03746d4f9f52fc6dd]
module.vpc.aws_vpc.main: Destruction complete after 0s

Destroy complete! Resources: 8 destroyed.

```